

GETTING STARTED GUIDE

Build your OS on Claude

Everything from the session, written down so you can follow along at your own pace, or find your way back if you get lost.

This is the guide we walk through together on the call. It is not homework, and there is nothing to read before we start.

Afterwards, keep it next to you. Read the step you are on, do it, then come back.

What is in this guide

1. What we are building, in plain words
2. Before you start: the setup checklist
3. The session, step by step (five steps)
4. Step 1: Open the kit in VS Code
5. Step 2: Onboard your OS
6. Step 3: Ask it the first real question
7. Step 4: Run your first audit
8. Step 5: Back it up to GitHub, and connect Vercel
9. How to get your GitHub token
10. How to get your Vercel token
11. Level up: your weekly ritual from Day 14
12. Your first two weeks
13. If you get lost
14. Words you will hear

How to use this guide

Each step tells you exactly what to click and exactly what to type. Anything you should type or paste into Claude is shown in a grey box like this:

```
/onboard
```

Anything in **orange** inside a grey box is a placeholder you replace with your own value. Everything else you can copy word for word.

THE ONE IDEA TO HOLD ON TO

Your OS is just a folder. Claude reads it at the start of every conversation. The better the folder describes you, the better Claude works for you. Everything we do today is about filling that folder well.

NOT HOMEWORK

We go through this guide together, live, on the call. You do not need to read ahead. The only preparation is the setup checklist in section 2, which was in the email.

USING CHATGPT INSTEAD OF CLAUDE?

The kit works with OpenAI's Codex extension too. Wherever this guide says "the Claude Code panel", use the Codex panel. Wherever it says to type `/onboard` , `/audit` or `/level-up` , type `$onboard` , `$audit` or `$level-up` instead. Everything else is identical.

1. What we are building, in plain words

Most people use Claude like a very smart stranger. Every conversation starts from zero. You explain who you are, what you are working on, what you have already decided, and then you get to your actual question.

An **OS**, the way we mean it here, fixes that. It is a folder on your laptop that holds:

- **Who you are** and who you work with or sell to.
- **What matters** for the next 90 days.
- **How you sound**, from real things you have written.
- **What you have decided**, and why.
- **What Claude can reach**: your calendar, inbox, task list, and so on, once you connect them.
- **What Claude knows how to do** for you: small reusable jobs called skills.

Claude reads this folder automatically every time you open it in VS Code. So instead of a smart stranger, you get something closer to a colleague who has worked with you for a year.

The four layers

Your OS has four layers. You will see these names again when you run the audit.

LAYER	THE QUESTION	YOU KNOW IT IS WORKING WHEN
Context	Does it know me?	A fresh conversation can say what you do and what matters this quarter without you explaining.
Reach	Can it touch my stuff?	"What is on my calendar tomorrow?" gets a real answer, not a request to paste it.
Skills	Does it know how to do my work?	A short command like <code>/weekly-review</code> runs a multi-step job.
Rhythm	Does it run without being asked?	Something useful happens on a schedule while you are away.

Today we build the first layer, Context, properly. The other three grow week by week using two tools that ship with the kit: `/audit` and `/level-up`.

The three tools in the kit

TOOL	WHAT IT DOES	WHEN
/onboard	Seven questions. Fills in your OS for the first time.	Today
/audit	Scores your OS out of 100 and names the three most valuable gaps.	Today, then weekly
/level-up	Finds one thing worth automating, scopes it, builds it.	From Day 14, then weekly

2. Before you start: the setup checklist

You should have done all of this before the session. If any line is not true for you yet, do it now. It takes about 20 minutes in total.

Your laptop

- Mac running macOS 13 or newer, or Windows 10 (version 1809 or later) or Windows 11.
- You can install software on it. At least 4 GB of RAM and 2 GB of free space.
- A stable internet connection. No iPads or phones for this.

Your AI account: pick one

Claude is what the kit is built for and what you will see on screen. If you are undecided, choose Claude.

2.1 CLAUDE (PREFERRED)

- **Claude Max**, the \$100 per month plan, is the real minimum.
- The free plan does not work with Claude Code at all.
- Claude Pro at \$20 technically works but runs out fast. You will hit the usage limit partway through the hour.
- Upgrade at claude.ai under Settings, then Plan. A company Team or Enterprise plan with Claude Code enabled also works.

2.2 CHATGPT (THE ALTERNATIVE)

- The same kit runs with OpenAI's **Codex** extension for VS Code.
- A free ChatGPT account signs in but its Codex allowance is too small to get through the hour.
- ChatGPT Plus at \$20 runs out fast, same as Claude Pro.
- **ChatGPT Pro**, from \$100 per month, is the real minimum. Upgrade at chatgpt.com under Settings, then Upgrade plan.

Software to install

- 1 **VS Code** from code.visualstudio.com. Mac: drag it into Applications. Windows: run the installer with the defaults.
- 2 **The AI extension** inside VS Code. Install the one that matches your account, not both. Open VS Code, press `Cmd+Shift+X` (Mac) or `Ctrl+Shift+X` (Windows), then:
 - **Claude:** search for **Claude Code** and install the one by **Anthropic**. Click the spark icon at the top right of the editor and sign in with your Claude account in the browser window that opens.
 - **ChatGPT:** search for **Codex** and install "**Codex – OpenAI's coding agent**" by **OpenAI**. Click the Codex icon in the left sidebar and sign in with your ChatGPT account. If you cannot see the icon, press `Cmd+Shift+P` / `Ctrl+Shift+P` and run "Codex: Open Codex Sidebar".

3

Git. Mac: open Terminal, type `git --version`, press Enter, and click Install if it offers.

Windows: install from git-scm.com/download/win with all defaults. This also gives you Git Bash, which Claude uses behind the scenes.

Accounts to create (both free)

- **GitHub** at github.com. Verify your email. Your OS will live here as a private backup.
- **Vercel** at vercel.com. Choose the Hobby plan and sign up with your GitHub account. Anything you build later gets hosted here.

Things to have ready

- **Two things you wrote recently**, saved in a note or text file. An email, a message, a post. Unedited. Your OS learns your voice from these.
- Your two or three priorities for the next 90 days.
- The one task that eats your week.

EVERYTHING HAPPENS INSIDE VS CODE

You will not use a separate command-line tool. The Claude Code extension (or the Codex extension) gives you a chat panel inside VS Code, and that panel is where you type every command in this guide.

3. The session, step by step

One hour. Five steps. Here is the shape of it, so you always know where you are.

- Step 1** **Open the kit in VS Code.** Get the starter folder onto your laptop and open it with Claude Code.
- Step 2** **Onboard.** Type `/onboard` and answer seven questions. This fills your OS.
- Step 3** **Ask the first real question.** "What should I focus on this week?" and see it answer like it knows you.
- Step 4** **Audit.** Type `/audit` and get your first score out of 100.
- Step 5** **Back it up.** Give Claude your GitHub token so your OS lives in a private repository, then your Vercel token for hosting later.
- Then** Questions. And you are done. Go and keep exploring.

IF YOU FALL BEHIND DURING THE SESSION

Do not panic and do not try to catch up live. Note which step you are on, keep listening, and use this guide afterwards. Every step is written to be done alone.

4. Step 1: Open the kit in VS Code

The starter kit is a small folder of files on GitHub. You are going to copy it onto your laptop and open it in VS Code. In Git language, copying a repository is called **cloning**.

The address you need

```
https://github.com/Xyz7765007/personal-os-starter
```

Clone it from inside VS Code

- 1 **Open VS Code.** If a folder is already open, go to File, then Close Folder, so you are on the Welcome screen.
- 2 **Open the command palette.** Press `Cmd+Shift+P` on Mac or `Ctrl+Shift+P` on Windows. A search box appears at the top.
- 3 **Type `Git: Clone`** and press Enter.
- 4 **Paste the address** above into the box and press Enter.
- 5 **Choose where to put it.** A folder picker opens. Pick somewhere you will find again, for example your Documents folder. Click "Select as Repository Destination".
- 6 **Click "Open"** when VS Code asks whether to open the cloned repository. If it asks whether you trust the authors, click "Yes, I trust the authors".

You should now see a list of files on the left: `CLAUDE.md`, `README.md`, `os-intake.md`, a `context` folder, and so on. That folder is your OS.

Open Claude Code on this folder

- 1 **Click the spark icon** at the top right of the editor, or press `Cmd+Esc` on Mac / `Ctrl+Esc` on Windows. The Claude Code panel opens.
- 2 **Check you are signed in.** If it asks you to sign in, a browser window opens. Sign in with your Claude account and come back.
- 3 **Say hello.** Type `hi, what is this folder?` and press Enter. Claude should describe the OS starter kit back to you. That proves it is reading the folder.

Using Codex? Click the Codex icon in the left sidebar instead of the spark icon, or run "Codex: Open Codex Sidebar" from the command palette. Then say hello the same way.

PERMISSION POP-UPS

The first time Claude wants to edit a file or run a command, the panel asks you to allow it. That is normal and it is a good thing. Read what it wants to do, then click Allow. If you are comfortable, you can choose the option that stops it asking again for the rest of the session.

IF GIT: CLONE DOES NOT WORK

Open the address above in your browser, click the green **Code** button, then **Download ZIP**.
Unzip it, rename the folder to `my-os`, then in VS Code go to File, Open Folder, and pick it.
Everything else in this guide works the same.

5. Step 2: Onboard your OS

This is the heart of the session. Claude asks you seven questions, writes your answers into a file called `os-intake.md`, and then generates the rest of your OS from them. About 15 minutes.

Start it

In the Claude Code panel, type this and press Enter:

```
/onboard
```

Using Codex? Type `$onboard` instead.

Claude will explain what is about to happen and then ask the first question. Answer in the chat like you are talking to a person. One question at a time.

The seven questions

#	QUESTION	TIPS FOR A GOOD ANSWER
1	Who are you, what do you do, and who do you do it for?	Name, role, what you make or run, who it is for. A few sentences.
2	Paste one or two things you wrote recently. Do not edit them.	Paste, do not type. Copy from your note and paste it in. See the box below.
3	What are your two or three biggest priorities for the next 90 days?	Use a number, a date or a deliverable. "Grow" will get pushed back on.
4	Where do the numbers that matter to you live?	Name the tools: a spreadsheet, an invoicing app, a dashboard.
5	Where do you talk to people day to day?	Gmail or Outlook, Slack or Teams, WhatsApp, and so on.
6	Where do meeting notes, recordings and documents live?	Note-takers, recorders, Drive, Notion, and so on.
7	What is the one task that eats your week, and where do you track work?	Be honest about the task. Name the to-do tool if you have one.

THE ONE RULE: PASTE YOUR WRITING SAMPLES

Question 2 is the only question with a hard rule. If you start typing a fresh sample into the chat, Claude will stop you and ask you to paste instead. That is on purpose. Writing done while you are mid-conversation with Claude already sounds like Claude. Copy something real from your inbox or your posts, paste it unedited, and move on.

What Claude creates at the end

When the seventh answer is in, Claude writes a set of files in one go. You will see them appear in the file list on the left.

FILE	WHAT IT HOLDS
<code>context/about-me.md</code>	Who you are, your role, the task that eats your week.
<code>context/about-work.md</code>	What you make or run, who it is for, where results are tracked.
<code>context/priorities.md</code>	Your 90-day priorities as a numbered list.
<code>references/voice.md</code>	Your writing samples, word for word.
<code>connections.md</code>	The tools you named, listed as "not yet connected". You wire them later.
<code>CLAUDE.md</code>	Your operating manual. Claude reads it at the start of every conversation.

Then it prints three lines telling you what to do now, tomorrow, and on Day 7. That is your cue for Step 3.

CHANGED YOUR MIND ABOUT AN ANSWER?

Open `os-intake.md` in VS Code, edit the answer, save, and type `/onboard` again. Claude refreshes only the files that changed and keeps a backup of the old ones in `archives/`. You can do this any time, forever.

6. Step 3: Ask it the first real question

This is the moment the folder stops being a folder. Type this into the Claude Code panel:

```
What should I focus on this week?
```

A good answer has three things in it:

- Three bullets that name **your actual priorities** from question 3, not generic advice.
- It sounds a little like **you**, because it read your writing samples.
- It ends by picking **one thing for Monday**, with a reason, and asks where it could take work off your plate entirely.

If the answer is generic, something in the intake is thin. Open `os-intake.md`, add more detail to question 1 or 3, and run `/onboard` again.

Try two more, if there is time

```
Draft a short reply to a client who asked for a call next week. Match my voice.
```

```
Log a decision: I am going to [something you decided today], because [why].
```

The second one appends an entry to `decisions/log.md`. Get into the habit. A decision without the reason is trivia; a decision with the reason is memory.

7. Step 4: Run your first audit

The audit scores your OS out of 100 across the four layers and tells you the three gaps most worth fixing. It reads your files; it never changes them. Type:

```
/audit
```

Using Codex? Type `$audit` instead.

How to read the report

You get a scoreboard like this:

Score: 32/100 (Foundation)			
Context	####	20/25	Strong
Reach	#	2/25	Missing
Skills	##	10/25	Thin
Rhythm	#	0/25	Missing

- **Context** should already be Solid or Strong. That is what onboarding did.
- **Reach** will be near zero. Nothing is connected yet. Expected.
- **Skills** starts at 10 because the kit ships three skills. It climbs when you build your own.
- **Rhythm** starts at zero or near it. It comes last, on purpose.

Below the scoreboard you get your **top three gaps**, each with one concrete next step, and a single line that says **Do this next**. That line is the only thing you need to act on this week.

SCORE	STAGE	WHAT IT MEANS
0 to 39	Foundation	Where almost everyone starts. Context is in, nothing is wired.
40 to 69	Built	Two or three tools connected, first skill of your own.
70 to 89	Compounding	It runs jobs for you and some of them happen on a schedule.
90 to 100	Autonomous	Rare. Do not chase it. Chase the next gap.

At the end Claude asks whether to save the report to `audits/` . Say yes. Next week's score only means something next to this week's.

RUN IT EVERY WEEK

Friday is a good day. Same command, two minutes, one number that should go up. If it does not go up, the top gap tells you why.

8. Step 5: Back it up to GitHub, and connect Vercel

Right now your OS lives only on your laptop. This step moves it into a private repository on GitHub so it is backed up after every session, and gives Claude a Vercel key so it can host things you build later. Two prompts, one key each.

First, the GitHub prompt

You need a GitHub personal access token. If you do not have one yet, follow section 9 and come back. Then paste this into the Claude Code panel, with your token where the orange text is:

```
<paste your GitHub token here> I am using a local setup, this is my GitHub access token. Add it to a .env file and host my OS in a private GitHub repository. Make sure that after every task or session you push everything to GitHub and update memory, decisions and learnings.
```

What Claude does with that:

- Creates a file called `.env` and puts the token in it. That file is ignored by Git, so the token never leaves your laptop.
- Creates a **private** repository on your GitHub account and pushes your OS into it.
- From now on, at the end of every session, it commits and pushes what changed, and updates `memory/learnings.md` and `decisions/log.md`.

Check it worked: open github.com in your browser, click your profile picture, then "Your repositories". You should see a new private repository with your OS files in it.

Then, the Vercel prompt

You need a Vercel token. Section 10 shows how to get one. Then paste:

```
<paste your Vercel token here> Here is my Vercel token. Save it in the .env file and use it for hosting anything going ahead.
```

Claude adds a second line to `.env`. Nothing else happens yet. The next time you build something with a web page in it, Claude can put it online with that key.

TOKENS ARE PASSWORDS

- Anyone with your token can act as you on that service. Do not paste it anywhere except the Claude Code panel on your own laptop.
- Both services show a token **once**. If you lose it, delete it and make a new one. You never need the old value.
- If you ever think a token leaked, go back to the token page on GitHub or Vercel, delete it, and create a fresh one. Then paste the fresh one to Claude with the same prompt.
- Claude will only ever store tokens in `.env`. If you see one anywhere else, tell Claude to move it.

Note: when you cloned the starter kit, the folder still remembers the starter kit's address as its origin. The GitHub prompt replaces that with your own private repository. That is expected.

9. How to get your GitHub token

A personal access token is a long password that lets Claude create repositories and push to them on your behalf. We use the **classic** kind, with every scope ticked, so Claude is never blocked by a missing permission later.

- 1 **Go to `github.com` and sign in.**
- 2 **Click your profile picture** in the top right corner, then click **Settings**.
- 3 **Scroll to the bottom of the left menu** and click **Developer settings**.
- 4 **Click Personal access tokens**, then **Tokens (classic)**.
- 5 **Click Generate new token**, then **Generate new token (classic)**. GitHub may ask for your password or a two-factor code. That is normal.
- 6 **In the Note box, type** `My OS` **so you remember what it is for.**
- 7 **Choose an expiration.** 90 days is a sensible default. "No expiration" means you never have to redo this, at the cost of a token that lives forever. Your call. If it expires, you just make a new one and paste it to Claude again.
- 8 **Select scopes: tick every box.** Tick each top-level checkbox in the list (`repo` , `workflow` , `write:packages` , and all the rest, down to the bottom). Ticking a top-level box ticks everything under it.
- 9 **Scroll down and click Generate token.**
- 10 **Copy the token immediately.** It starts with `ghp_` . Click the copy icon next to it. You will not see it again after you leave this page.

Now go back to Step 5 and paste it into the GitHub prompt.

LOST IT ALREADY?

No problem. On the same Tokens (classic) page, click Delete next to "My OS" and repeat from step 5. Old tokens can be thrown away freely.

10. How to get your Vercel token

Vercel is where anything you build with a web page will be hosted. The token lets Claude deploy on your behalf. We give it **full account** scope so it can create projects freely.

- 1 **Go to vercel.com and sign in.** If you have not signed up, do it now with your GitHub account and choose the free Hobby plan.
- 2 **Click your profile picture** in the top right corner, then click **Account Settings**.
- 3 **In the left menu, click Tokens.**
- 4 **Click Create.**
- 5 **Token name:** type `My 0S`.
- 6 **Scope:** choose **Full Account**.
- 7 **Expiration:** pick what you are comfortable with. Same trade-off as GitHub. A new one takes one minute to make.
- 8 **Click Create.** The token appears once. Click the copy icon.

Now go back to Step 5 and paste it into the Vercel prompt.

11. Level up: your weekly ritual from Day 14

The audit tells you whether your OS is *built right*. Level up asks a different question: *what could it do for me that it cannot yet?* Each run finds one thing worth automating, scopes it properly, and builds it. One run, one shipped thing.

WAIT UNTIL DAY 14

Run it for the first time about two weeks after onboarding, once you have used your OS for real questions and connected at least one tool. Earlier than that, the answers are too generic to be worth building.

To start it, type:

```
/level-up
```

Using Codex? Type `$level-up` instead.

Part 1: Find

Claude asks five questions about your week, one at a time. You answer honestly.

- What did you do three or more times?
- What felt manual, boring, or copy-paste?
- What made you think "a smart intern could do this"?
- If ten times the work showed up tomorrow, what would break first?
- What would get you ten times the results tomorrow?

It cross-checks your answers against your priorities and the task that eats your week, and gives you one to three candidates. You pick one.

Part 2: Scope

Five short questions about the candidate you picked. Any of them can end the run early, and that is a good outcome, not a failure.

1. **Which bottleneck does this remove?**
2. **Stop it, automate it, or hand it off?** Always in that order. If nothing breaks when you stop doing it, stop doing it. Claude logs the decision and you are done for the week.
3. **Map it in five parts.** What triggers it, where the information comes from, how it changes shape, where it branches, where the output goes. If you cannot describe it, sketch it on paper first.
4. **Pick the autonomy level.** Suggested, drafted, supervised, or fully autonomous. Claude defaults to the lowest level that works and will push back if you ask for fully autonomous on a first build.
5. **Which number does it move?** Time saved, response time, errors, conversions. No number, no build.

Claude writes the whole spec into `decisions/log.md` , so you have a record of what you decided and why.

Part 3: Build

Claude asks how you want to ship it, from simplest to most complex: a saved prompt, a skill that runs a script, a skill with one AI step, or an agent. It recommends the simplest option that solves the problem. You have to choose more on purpose.

Whatever gets built carries a line in it that says `maturity: manual-first` . That is a reminder to run it by hand three times before you ever put it on a schedule.

WHY THIS IS THE REAL OUTCOME

After five or six weeks of this, you will catch yourself noticing automation candidates mid-week without being asked. The folder is where the OS lives. The habit is the OS.

12. Your first two weeks

WHEN	DO	WHY
Today	<code>/onboard</code> , first question, <code>/audit</code> , both tokens	Context layer done and backed up.
Tomorrow	Open <code>connections.md</code> , pick one row, ask Claude to wire it	Reach grows one tool at a time. Pick the one that feeds your biggest priority.
Days 3 to 6	Bring real questions. Draft real emails. Log real decisions.	The OS gets better every time you use it for something true.
Day 7	<code>/audit</code>	Second score. Act on the "Do this next" line.
Day 14	<code>/level-up</code>	First automation.
Every Friday	<code>/audit</code> then <code>/level-up</code>	Score, then ship one thing.

Prompts that work well in week one

Wire up my calendar. Tell me what you need from me.

Read my priorities and tell me which one I am neglecting, based on what I have asked you about this week.

Write a reference note for [tool] in `references/` so you never have to figure out its API again.

What did we decide last week and why?

Five habits that keep it healthy

1. **Log the why.** Every meaningful decision goes in `decisions/log.md` with the reasoning.
2. **Research once.** The day you wire a tool, have Claude write `references/<tool>-api.md`.
3. **Stop before you automate.** If nothing breaks when a task stops, stop the task.
4. **Lowest autonomy that works.** Suggest before draft, draft before run, run before autonomous.
5. **Push after every session.** Your OS should survive your laptop.

13. If you get lost

I cannot see the Claude Code panel (or the Codex panel)

Press `Cmd+Shift+P` (Mac) or `Ctrl+Shift+P` (Windows), type `Claude Code` (or `Codex: Open Codex Sidebar`), and pick the option that opens it. If nothing shows up, the extension is not installed: go to Extensions, search for Claude Code by Anthropic (or Codex by OpenAI), install, and restart VS Code.

It keeps asking me to sign in

Make sure the browser window that opened finished signing you in and showed a success message. If the browser did not open at all, look in the Claude panel for a link or a code to copy. If it still loops, sign out of `claude.ai` in your browser, sign back in, then try again in VS Code.

It says my plan does not support Claude Code, or I hit a usage limit

Claude: you are on the free plan, or on Pro and the allowance ran out. Upgrade to Max at `claude.ai` under Settings, then Plan, and sign in again. ChatGPT: a free or Plus account has run out of Codex allowance. Upgrade to Pro at `chatgpt.com` under Settings, then Upgrade plan.

Git: Clone says "git not found"

Git is not installed. Mac: open Terminal, type `git --version`, click Install. Windows: install from `git-scm.com/download/win` with the defaults. Then restart VS Code. Or use the Download ZIP route in Step 1.

I typed `/onboard` and nothing happened

Check the file list on the left shows a `.claude` folder with `skills` inside it. If it does not, you opened the wrong folder or a folder inside the kit. Go to File, Open Folder, and pick the folder that contains `CLAUDE.md`. If the folder is right, start a new conversation in the panel and try again.

Claude keeps asking permission for everything

That is the default and it is safe. When it asks, you can pick the option that allows the same kind of action for the rest of the session. It will still ask for anything new.

The answer to "what should I focus on this week" was generic

The intake is thin. Open `os-intake.md`, add detail to questions 1, 3 and 7, save, and type `/onboard` again.

I pasted a token and Claude put it in the wrong place

Tell it: Move that token into `.env` and remove it from everywhere else. Then, to be safe, delete the token on GitHub or Vercel, make a fresh one, and paste it again.

I want to start over completely

Delete the folder, and if you already created a private repository, delete that on GitHub too (repository Settings, scroll to the bottom, Delete this repository). Then do Step 1 again. It takes five minutes. Nothing is precious yet.

Something else

Ask Claude. Seriously. Type `I am lost. I was trying to [what you were doing]. What state is this folder in and what should I do next?` It can read the folder and tell you.

14. Words you will hear

OS

Your folder of context, connections, skills and habits that Claude reads at the start of every conversation. Not software. A folder.

VS Code

A free editor from Microsoft. We use it as the window that holds your folder and the Claude Code panel.

Claude Code

Claude working inside your folder: it can read and write files and run commands, with your permission. You use it through the VS Code extension.

Codex

OpenAI's equivalent of Claude Code, for ChatGPT accounts. Same idea, same kit, and you type `$onboard` instead of `/onboard`.

Skill

A small instruction file at `.claude/skills/<name>/SKILL.md`. You run it by typing `/name`. The kit ships three.

CLAUDE.md

Your operating manual. Claude reads it automatically. Onboarding fills it in.

Repository (repo)

A folder that Git tracks, so every change is saved with history. Your OS is one.

Clone

Copying a repository from GitHub to your laptop.

Commit and push

Commit saves a snapshot of your changes with a message. Push sends those snapshots to GitHub. Claude does both for you at the end of each session.

GitHub

A website that stores repositories. Your OS lives there privately as a backup.

Vercel

A hosting service. Anything you build with a web page can be put online there in one step.

Token

A long password that lets Claude act on your behalf on a service. Treat it like a password.

.env

The one file where tokens live. Git ignores it, so it never leaves your laptop.

Connection

A tool Claude can reach on your behalf: your calendar, inbox, task list. Listed in `connections.md`.

Audit

The weekly score out of 100. Run with `/audit`.

Level up

The weekly ritual that finds and builds one automation. Run with `/level-up`.

You made it to the end. Close this, open VS Code, and ask your OS something real.